



BACHARELADO EM CIÊNCIA DA COMPUTAÇÃO

GUILHERME FABRICIO BRITO DA ROSA
HARRISON DE CARVALHO ALVARENGA
RAUL SOARES DE CARVALHO

TRABALHO COMPILADORES

Análise Léxica

LAVRAS

2025

1. Introdução

A construção de compiladores é um processo fundamental na ciência da computação, permitindo a tradução de código escrito em linguagens de alto nível para código de máquina executável. A primeira fase desse processo é a análise léxica, responsável por ler o fluxo de caracteres do código-fonte e agrupá-los em unidades significativas chamadas tokens.

Este relatório descreve a implementação de um analisador léxico para uma linguagem de programação hipotética denominada C-, utilizando a ferramenta Flex (Fast Lexical Analyzer Generator). O objetivo principal foi criar um analisador capaz de reconhecer os diferentes componentes léxicos da linguagem C- (palavras-chave, identificadores, números, operadores, pontuação), ignorar espaços em branco e comentários, e reportar erros léxicos encontrados, indicando a linha e coluna onde ocorreram.

2. Referencial Teórico

2.1 Análise Léxica

A análise léxica é a fase inicial do processo de compilação. O analisador léxico, também conhecido como scanner, lê o código-fonte caractere por caractere e o divide em uma sequência de tokens. Cada token representa uma unidade léxica da linguagem, como uma palavra-chave (if, while), um identificador (variavel, func), um número (123, 3.14), ou um operador (+, =). Lexema: Sequência de caracteres no código-fonte que corresponde a um padrão de um token. Por exemplo, if, contador, 100, +. Token: Par composto por um nome (ou tipo) de token e, opcionalmente, um valor de atributo. Exemplos: (KEYWORD, "if"), (IDENTIFIER, "contador"), (INTEGER, 100), (OPERATOR, "+"). Padrão: Descrição da forma que os lexemas de um token podem assumir, geralmente especificada por expressões regulares.

2.2 Expressões Regulares

Expressões regulares são uma notação formal para especificar padrões de strings. Elas são amplamente utilizadas para definir os padrões dos tokens em análise léxica. Operações comuns incluem concatenação, união (|), e fecho de Kleene (*, zero ou mais ocorrências; +, uma ou mais ocorrências; ?, zero ou uma ocorrência).

2.3 Diagrama de Transição

Foram feitos dois diagramas de transição: o primeiro (Automato_Completo.jff.jpg) contém um único autômato com todas as transições, porém com baixa organização visual devido a várias transições que incidem em um mesmo estado. O segundo (Automato_Desmembrado.jff.jpg) contém nove autômatos, mas que representam apenas um, de tal modo que podemos observar dois nomes de estado repetindo neles (q0 representa o mesmo estado inicial, e q27 representa o mesmo estado). Também há um estado adicional (q13). Essas diferenças são apenas para melhorar a organização visual,

visando o melhor entendimento do diagrama. A ferramenta utilizada para criar o diagrama foi o JFLAP.

2.4 Flex (Fast Lexical Analyzer Generator)

Flex é uma ferramenta que gera analisadores léxicos (scanners) em linguagem C a partir de um arquivo de entrada (.l) que descreve os padrões dos tokens usando expressões regulares e as ações C a serem executadas quando cada padrão é reconhecido. A estrutura básica de um arquivo .l é:

```
C/C++
%{
    // Definições C (includes, variáveis globais)
}%
/* Definições de nomes para expressões regulares */
%%
/* Regras: Padrão { Ação C } */
%%
/* Código C adicional (funções auxiliares, main) */
```

3. Descrição do Trabalho

O analisador léxico para a linguagem C- foi implementado utilizando a ferramenta Flex, seguindo a estrutura padrão de um arquivo .l.

3.1 Estratégias de Solução

a. Definições Globais (%{ ... %}):

Nesta seção, foram incluídos os cabeçalhos C necessários (stdio.h, string.h) e declaradas variáveis globais para rastrear o número da linha (line_number, inicializada com 1), o número da coluna (column_number, inicializado com 1) e a contagem de erros (errors_count). A variável yyout foi definida para direcionar a saída para stdout.

b. Definições de Expressões Regulares:

Foram definidos nomes para expressões regulares frequentemente usadas, como digit, letter, integer_num, float_num, identifier, ws (whitespace), newline, etc. Isso melhora a legibilidade das regras. A expressão para float_num ({digit}+(\.{digit}+)?([Ee][+-]?{digit}+)?) foi definida para reconhecer números com

parte decimal e/ou expoente opcionais. A definição `array_identifier` foi criada para reconhecer identificadores seguidos por um índice inteiro entre colchetes.

c. Estado de Comentário (%x COMMENT):

Foi definido um estado exclusivo (COMMENT) para lidar corretamente com comentários de múltiplas linhas (`/* ... */`), garantindo que o conteúdo do comentário seja lido e descartado, sem ser interpretado como tokens, enquanto se atualiza corretamente os números de linha e coluna.

d. Regras Léxicas (%% ... %%):

Esta seção contém o mapeamento entre os padrões (expressões regulares ou strings literais) e as ações C.

- **Comentários:** A regra `/*` ativa o estado COMMENT. Dentro deste estado, regras específicas consomem o conteúdo do comentário, incluindo novas linhas (atualizando `line_number` e resetando `column_number`), até que o padrão `*/` seja encontrado, retornando ao estado inicial (INITIAL). A saída (`fprintf`) dentro das regras de comentário parece ter sido usada para depuração, mostrando o conteúdo do comentário sendo processado.
- **Palavras-Reservadas:** Strings literais como `int`, `float`, `if`, `while`, etc., são reconhecidas diretamente. A ação associada imprime o número da linha, coluna, o lexema (`yytext`) e o tipo do token (ex: PALAVRA RESERVADA int). A coluna é atualizada (`column_number += yyleng`).
- **Operadores e Pontuação:** Strings literais para operadores (`+`, `-`, `*`, `/`, `=`, `==`, `!=`, etc.) e pontuação (`(`, `)`, `{`, `}`, `;`, etc.) são tratadas de forma similar às palavras-chave.
- **Números:** As regras `{float_num}` e `{integer_num}` reconhecem números reais e inteiros, respectivamente, usando as definições prévias. A ação imprime as informações do token e atualiza a coluna. Observação: A ordem destas regras é importante; `{float_num}` deveria idealmente vir antes de `{integer_num}` para garantir o reconhecimento correto de números que podem ser interpretados como ambos (ex: 123.45 poderia ser reconhecido como o inteiro 123 se a regra de inteiro vier primeiro).
- **Identificadores:** A regra `{identifier}` reconhece identificadores válidos. A regra `{array_identifier}` reconhece especificamente identificadores de vetores. Observação: A regra `{letter}` isolada pode causar conflitos com `{identifier}`, pois um identificador começa com uma letra. A regra `{array_identifier}` também pode precisar de ajuste de ordem em relação a `{identifier}`.
- **Espaços em Branco e Novas Linhas:** A regra `{ws}` simplesmente atualiza o número da coluna para ignorar espaços e tabs. A regra para newline (não explicitamente visível no trecho, mas necessária) deveria incrementar `line_number` e resetar `column_number`.

- **Erros Léxicos:** A regra {other} captura qualquer caractere que não corresponda a nenhuma das regras anteriores. Ela imprime uma mensagem de erro indicando o caractere inválido, a linha e a coluna, e incrementa errors_count.
- e. Código Auxiliar (%% ... %% após regras):
- Contém a função main, que abre o arquivo de entrada fornecido como argumento (argv[1]), atribui o ponteiro do arquivo a yyin, chama yylex() em loop (implicitamente, pois yylex() é chamado até o fim do arquivo) para processar a entrada, e fecha o arquivo. A função yywrap() retorna 1, indicando que não há mais arquivos de entrada a processar.

3.2 Geração e Compilação

O analisador léxico é gerado executando flex c-minus-lexer.l, que produz o arquivo lex.yy.c. Este arquivo é então compilado usando gcc lex.yy.c -lfl -o c-minus-lexer para criar o executável.

4. Testes Executados e Resultados Obtidos

Para validar o analisador léxico, foram criados diversos arquivos de teste (.txt) contendo:

- Código C- válido com diferentes tipos de tokens (palavras-chave, identificadores, números inteiros e reais, operadores, pontuação).
- Código contendo comentários de múltiplas linhas.
- Código contendo espaços em branco e múltiplas linhas.
- Código contendo erros léxicos (caracteres inválidos como @, \$).

O analisador foi executado com cada arquivo de teste usando o script scriptCompileAndRun.sh ou manualmente (./c-minus-lexer teste.txt).

```
C/C++
/* Calcula fatorial */
int fact(int n) {
    if (n <= 1) return 1;
    else return n * fact(n-1);
}

float pi = 3.14;
int count = 0;
```

Resultado esperado (Saída do Analisador):

Unset

1(1): /*...*/ (COMENTARIO)

2(1): int (PALAVRA RESERVADA int)

2(5): fact (IDENTIFICADOR)

2(9): ((ABRE PARENTESSES (

2(10): int (PALAVRA RESERVADA int)

2(14): n (IDENTIFICADOR)

2(15):) (FECHA PARENTESSES))

2(17): { (ABRE CHAVES {)

3(3): if (PALAVRA RESERVADA if)

3(6): ((ABRE PARENTESSES (

3(7): n (IDENTIFICADOR)

3(9): <= (MENOR IGUAL <=)

3(12): 1 (NUMERO INTEIRO)

3(13):) (FECHA PARENTESSES))

3(15): return (PALAVRA RESERVADA return)

3(22): 1 (NUMERO INTEIRO)

3(23): ; (PONTO E VIRGULA ;)

4(3): else (PALAVRA RESERVADA else)

4(8): return (PALAVRA RESERVADA return)

4(15): n (IDENTIFICADOR)

4(17): * (VEZES *)

4(19): fact (IDENTIFICADOR)

4(23): ((ABRE PARENTESSES (

4(24): n (IDENTIFICADOR)

4(25): - (MENOS -)

4(26): 1 (NUMERO INTEIRO)

4(27):) (FECHA PARENTESSES))

4(28): ; (PONTO E VIRGULA ;)

5(1): } (FECHA CHAVES })

7(1): float (PALAVRA RESERVADA float)

7(7): pi (IDENTIFICADOR)

7(10): = (ATRIBUICAO =)

```
7(12): 3.14 (NUMERO REAL)
7(16): ; (PONTO E VIRGULA ;)

8(1): int (PALAVRA RESERVADA int)
8(5): count (IDENTIFICADOR)
8(11): = (ATRIBUICAO =)
8(13): 0 (NUMERO INTEIRO)
8(14): ; (PONTO E VIRGULA ;)
```

Exemplo de Entrada com Erro (teste_erros.txt):

```
C/C++
int val = 10@;
```

```
Unset
1(1): int (PALAVRA RESERVADA int)
1(5): val (IDENTIFICADOR)
1(9): = (ATRIBUICAO =)
1(11): 10 (NUMERO INTEIRO)
(1) Erro léxico na linha 1 e na coluna 13. Entrada -> "@"
1(14): ; (PONTO E VIRGULA ;)
```

Os testes demonstraram que o analisador léxico foi capaz de:

- Identificar corretamente todos os tipos de tokens definidos para a linguagem C-.
- Ignorar espaços em branco, tabulações e novas linhas.
- Processar e ignorar corretamente comentários de múltiplas linhas.
- Reportar erros léxicos, indicando o caractere inválido, a linha e a coluna correspondentes. Manter a contagem correta de linhas e colunas durante a análise.

5. Conclusão

A implementação do analisador léxico para a linguagem C- utilizando a ferramenta Flex foi concluída com sucesso. O analisador é capaz de segmentar o código-fonte em tokens, descartar elementos não relevantes como espaços e comentários, e identificar erros léxicos básicos, fornecendo informações de localização precisas.

Esta etapa representa a base fundamental para as fases subsequentes da compilação, como a análise sintática, que receberá a sequência de tokens gerada por este analisador. Possíveis melhorias futuras poderiam incluir o tratamento de outros tipos de comentários (como `/* */`) ou a otimização das regras para lidar com ambiguidades de forma mais robusta.

6. Referências Bibliográficas

1. AHO, Alfred V.; SETHI, Ravi; ULLMAN, Jeffrey D. Compilers: Principles, Techniques, and Tools. Addison-Wesley, 1986. (Livro do Dragão)
2. LEVINE, John R.; MASON, Tony; BROWN, Doug. lex & yacc. O'Reilly & Associates, Inc., 1992.
3. Documentação Oficial do Flex: <https://flex.sourceforge.io/manual/>
4. Site oficial do JFLAP: <https://jflap.org/>